

캡스톤디자인과창업프로젝트B

<Learning Village>

AI가 영어 공부 자료를 분석해
초/중등 학생에 대한 맞춤형 게임을
만들어주는
교육용 게임 웹사이트

팀명	200OK
트랙	산학 프로젝트
지도교수	오유란 교수님
팀장	2371011 김규린
팀원	2373040 김연우
	2272027 심희윤

<목차>

I. Team Info.....	2
A. 과제명.....	2
B. 팀 정보.....	2
C. 팀 구성원 및 역할.....	2
II. Project Summary.....	3
A. 문제 정의.....	3
B. 기존 연구 및 서비스 비교.....	4
C. 제안 내용.....	5
D. 기대 효과 및 의의.....	6
E. 주요 기능 리스트.....	7
III. Project Design.....	10
A. 주요 기능 유스케이스.....	10
B. 요구사항 정의서.....	12
C. 전체 시스템 구성.....	14
D. 주요 엔진 및 기능 설계.....	14
E. 주요 기능의 구현.....	18
F. 기타 (AI 설계 상세 내용).....	22

I. Team Info

A. 과제명

AI가 영어 공부 자료를 분석해 초/중등 학생에 대한 맞춤형 게임을 만들어주는 교육용 게임 웹사이트

B. 팀 정보

- 팀명: 200 OK
- 팀 번호: 34

C. 팀 구성원 및 역할

이름	역할	학번	담당 업무
김규린	팀장	2371011	• 전체 백엔드 API 시스템 구현 • Redis 기반 비동기 처리 환경 및 백엔드 서버 인프라 배포 • JUnit 활용 테스트 코드 작성을 통한 API 무결성 검증 • 게임 그래픽 및 로직 일부 구현
김연우	팀원	2373040	• Grammar-aware RAG 기반 AI 문제 생성 파이프라인 및 Frontend-Backend-AI Server 간 자동화 데이터 처리 구조 설계 및 최적화 • NLP 기반의 정밀한 문장 분석 및 문제 생성 로직 구현 • Celery와 Redis를 이용한 대용량 비동기 처리 구조 설계
심희윤	팀원	2272027	• 웹 페이지 UI/UX 디자인 구현 • 게임에 대한 그래픽 및 로직을 구현 • 웹 페이지 프론트엔드 구현 • 담당 화면별 기능 테스트 및 사용자 UI/UX 오류 수정

II. Project Summary

A. 문제 정의

프로젝트 기획 배경 및 필요성

현재 교육 현장에서는 디지털 전환과 함께 개별 맞춤형 학습의 중요성이 대두되고 있음. 선행 연구 및 교사 인터뷰에 따르면, 교사의 58.3%가 "디지털 전환의 핵심은 개별 맞춤형 학습 지원"이라고 응답했으나, 동시에 59.6%의 교사가 "맞춤형 수업 준비 및 자료 제작 환경의 부재"로 인해 엄청난 업무 부담을 겪고 있음.

또한, 학생들의 참여를 유도하기 위해 교육 현장에서 게임 요소가 있는 Kahoot! 등을 적극 도입하고 있으나, 학생들이 게임 자체에만 집중하는 주객전도 현상이 자주 일어나고 있음.

나아가, 이러한 문제를 시중에 나와있는 AI로 해결하려 할 때 기존 교과서나 교재 데이터를 무단 학습시키거나 활용할 경우, 저작권 침해 문제가 발생하여 서비스의 확장성이 크게 제한되기도 함.

따라서, 저작권 제약 없이 학생 개개인의 수준에 맞춘 고품질의 학습 콘텐츠를 실시간으로 제공하고, 게임과 학습이 완벽히 일치된 환경을 제공하는 교육용 웹페이지가 필요할 것이라 파악함.

Target Customer: 단순 반복 학습으로 이루어지는 영어 학습에 어려움을 느끼거나 지루함을 느껴, 영단어와 영문법을 쉽고 재밌게 배우고 싶은 초·중등 학생

해결하고자 하는 문제점 (Pain Points):

1. 학습 동기 저하 : 기존의 단순 반복형 영어 학습(단어 암기, 문법 풀이)은 학생들에게 쉽게 지루함을 주며 자기주도적 학습을 방해함.
2. 획일화된 학습 콘텐츠와 개인에 대한 맞춤형 학습의 부족 : 학생 개개인의 수준과 관심사에 맞는 세분화된 맞춤형 학습 자료를 실시간으로 대량 생성하기 어려움.

3. 취약점을 알 수 없는 결과 중심적 피드백 : 기존의 피드백 방식에서는 학습의 결과를 중심으로 학생의 학습 현황과 이해도를 측정하기 때문에, 학습 과정 수행 중에서 찾을 수 있는 취약점을 피드백에 반영하기가 어려움.

B. 기존 연구 및 서비스 비교

유사한 교육용 게임 및 문제 생성 시스템이 존재하지만, 본 프로젝트는 기존 서비스들이 가진 한계를 분석하여 그 한계를 기술적, 구조적으로 극복한 통합 솔루션을 제공하고자 함

기존 서비스들에 대해 분석한 결과:

구분	유사 과제 및 서비스	장점	단점 및 한계
게임형 학습 플랫폼	Kahoot, Quizlet 등	높은 학생 참여도 및 직관적인 인터페이스 제공	정적인 문제 콘텐츠, 개인 맞춤형 심화 학습의 어려움, 게임 플레이와 실제 학습 내용의 괴리 발생
LLM 문제 생성 시스템	일반 텍스트 기반 LLM	빠르게 대량의 문제 자동 생성 가능	할루시네이션으로 인한 문제 품질의 불균일, 높은 API 호출 비용, 교과서 데이터 활용 시 저작권 이슈 발생 가능성 존재
영어 학습 앱	워드마스터 학습앱, 해커스 보카 등	반복 학습을 통한 암기 효율성	단조로운 구조로 인한 학생들의 빠른 흥미 상실 및 자기주도 학습 동기 저하

기존의 에듀테크 서비스들은 각자의 장점에도 불구하고, 학습의 깊이, 지속적인 흥미, 안정적인 학습 콘텐츠 생성이라는 세 가지 요소를 동시에 충족시키지 못한다는 한계가 있음

- 게임형 플랫폼들은 학습자들의 초기 흥미를 끄는 데는 성공적이지만, 학습 내용 자체가 게임의 메커니즘과 직결되지 않아, 학습자들이 수업 내용에 대한 학습보다는 게임 자체에 몰두하는 현상이 발생함

- LLM을 도입한 최신 서비스나 기존 단어장 앱들을 통해 학습의 밀도를 높일 수 있으나, 저작권 이슈 및 막대한 유지 비용 문제가 발생하거나 학습자들의 장기적인 흥미 유발이 어려움

본 프로젝트가 기존 서비스들에 대해 갖는 차별점 및 효과:

- 1) 저작권 원천 차단 및 비용 절감: 단순 LLM에 의존하는 대신 Open Corpus(Wikitext 등)와 NLP(Stanza 기반 UD parsing)를 결합한 자체 AI 파이프라인을 구축함. 저작권 문제 없이 고품질의 문법 구조 기반 문제를 일관성 있게 생성할 수 있으며, 무분별한 API 호출 비용을 획기적으로 절감
- 2) 학습과 게임의 완전한 결합: Kahoot처럼 단순히 문제를 풀기만 하는 형태의 퀴즈 게임이 아닌, 메이드 인 와리오 스타일의 빠르고 직관적인 미니게임들을 도입함. 학습 내용 자체가 게임의 클리어 조건이 되도록 설계하여 재미와 순간 집중력을 극대화
- 3) 동적 맞춤형 피드백 제공: 정적인 문제은행 방식이 아닌, 게임 플레이 중 실시간으로 수집되는 사용자 데이터를 분석함. 가장 많이 틀리는 오답 유형을 파악하여 취약점 특화 게임을 자동 재생성하는 등의 피드백이 가능해져, 완벽한 개인 맞춤형 반복 학습을 구현

C. 제안 내용

본 프로젝트는 위에서 제시한 Pain Point들을 해결하고자, 다음과 같은 Solution들을 제안

Pain Point 1) 획일화된 학습 콘텐츠와 개인에 대한 맞춤형 학습의 부족

Solution : 사용자가 원하는 방식의 맞춤형 게임 제공

- 1) 사용자의 입력 데이터를 바탕으로 맞춤형 문제 제공 : 사용자가 원하는 학습 방향성과 내용을 입력하여, 사용자가 원하는 학습 방식에 맞게 문제를 제공
- 2) 학습 상황을 실시간으로 측정 및 활용 : 게임을 플레이하고 있는 중에도 사용자의 실력에 따라 게임 속도가 동적으로 조절되며 몰입감을 제공하고, 게임 플레이 중에도 학습 상황을 추정할 수 있는 데이터들을 수집 및 다음 게임 플레이시 활용

Pain Point 2) 취약점을 알 수 없는 결과 중심적 피드백

Solution : 다양한 방식의 취약점 분석 및 피드백

- 1) 리포트, 대시보드, 게임 재생성 기능 제공 : 게임 플레이 중 발견할 수 있는 취약점(가장 많이 틀리는 오답 유형 등)들에 대한 분석한 취약점 리포트, 게임 결과 대시 보드, 지금까지 수집한 정보를 토대로 생성할 수 있는 취약점 특화 게임 등을 제공

Pain Point 3) 공부 동기 부여 제공의 어려움

Solution : 게이미피케이션을 통한 동기 부여 제공

- 1) 게이미피케이션 : 메이드 인 와리오 게임 형식을 채용, 빠르고 짧으며 순간적인 집중력을 요구하는 여러가지 미니게임들을 통해 공부를 효율적으로 재밌게 할 수 있게 함. 또한 게임 플레이 보상으로 얻은 코인을 통해 다마고치형 육성 게임을 진행할 수 있음
- 2) 공유 기능 : 자신이 만든 게임을 공유하고 다른 사람이 만든 게임을 플레이할 수 있어 좀 더 빠르게 게임 플레이가 가능하며, 게임 제작에 대한 동기도 부여함

D. 기대 효과 및 의의

본 프로젝트의 결과물은 개인의 학습 환경에 긍정적인 변화를 가져오며, 기술적으로도 지속 가능한 에듀테크 모델을 제시함

- 1) 학습 몰입도 향상 및 자기주도 학습 실현 : 게이미피케이션과 AI 맞춤형 콘텐츠, 유저 간 게임 공유 기능을 통해 기존 학습 시스템 대비 초·중등 학생의 영어 학습 참여도와 성취도를 긍정적으로 향상시킴
- 2) 공부에 대한 제약 감소 : 학생이 교재, 학원, 교사의 개입 없이도 언제 어디서든 고품질의 맞춤형 학습을 수행할 수 있어, 양질의 교육에 대한 접근성을 높임
- 3) 저작권 제약 없는 확장형 파이프라인 구축 : 저작권 문제 없는 자동화 시스템 구축으로 AI 할루시네이션과 저작권 침해라는 두 가지 리스크를 동시에 해결하고, 콘텐츠 제작 비용을 획기적으로 절감하여 지속 가능한 교육 플랫폼 모델을 제시
- 4) 시스템의 확장성 : 현재는 영어 과목(단어/문법)에 집중하고 있으나, 구축된 NLP 분석 기반의 게임 생성 시스템은 추후 타 언어학습이나 구조 분석이 가능한 다른 과목으로도 쉽게 확장될 수 있는 강력한 기술적 토대가 될 수 있음

E. 주요 기능 리스트

Solution	필요 기술	결과	세부사항
사용자가 원하는 방식의	자체 제작 Grammar-aware	문제 생성	저작권 프리 데이터 기반의 AI 파이프라인을 통해 저작권 이슈 없어,

맞춤형 게임 제공	RAG Pipeline		지속적으로 안정적인 품질의 문제를 생성 STANZA 기반 문법 분석 및 Grammar Tag Retrieval 구조를 활용하여 사용자 조건에 맞는 corpus 문장을 검색하고, Retrieval 결과와 사용자 학습 데이터를 기반으로 LLM이 문제·해설·피드백을 JSON 형태로 생성하도록 하는 AI 문제 생성 파이프라인 구현
	GEMINI API	문제 해설 및 학습 내용 제공	외부 LLM을 연동하여 심도 있는 문제 해설과 풍부한 학습 보조 내용을 실시간으로 제공
	Postgresql	영속성 데이터 저장	PostgreSQL(RDBMS)을 통해 영속성 데이터를 안전하게 저장
	Spring Boot(Java)	사용자/게임/데이터 등 관리	사용자 정보, 게임 로직, 데이터 전반을 REST API와 트랜잭션 처리를 통해 안정적으로 관리
다양한 방식의 취약점 분석 및 피드백	자체 제작 Grammar-aware RAG Pipeline	결과 분석	자체 제작 AI를 사용해 높은 품질의 영단어/영문법 취약점 분석 및 리포트, 대시보드, 게임 재생성과 같은 피드백 기능을 구현 사용자 오답 패턴 및 문법 태그를 기반으로 취약 문법 유형을 분석하고 관련 corpus 문장을 Retrieval한 후, Retrieval 결과와 사용자 학습 데이터를 활용하여 LLM이 맞춤형 문제·해설·피드백을 JSON 형태로 생성하는 AI 분석 및 문제 생성 파이프라인 구현
	Postgresql	영속성 데이터 저장	PostgreSQL(RDBMS)을 통해 영속성 데이터를 안전하게 저장
	Spring Boot(Java)	사용자/게임/데이터 등 관리	사용자 정보, 게임 로직, 데이터 전반을 REST API와 트랜잭션 처리를 통해 안정적으로 관리
게이미피케이션을 통한 동기 부여 제공	React	사용자 UI 구현	단일 페이지 애플리케이션(SPA) 구조를 채택하여 끊임 없는 사용자 인터페이스를 제공하고 반응성을 높임
	Phaser.js	미니게임 구현	HTML5 게임 엔진을 활용해 학습에 흥미를 더할 수 있는 인터랙티브한, 메이드 인 와리오 LIKE 미니게임과 다마고치형 육성 게임 기능을 구현

F. 주요 개발 내용

1. 스타트

스타트 단계에서는 핵심 엔진 기능의 프로토타입 구현과 기술적 검증을 목표로 개발을 진행하였다.

1) AI 문제 생성 엔진

- PDF 기반 입력 데이터를 활용하여 문제를 자동 생성하는 AI 파이프라인 구축
- GPT API와 연동하여 문제·정답을 JSON 형태로 생성하는 구조 구현

→ 기본 문제 생성 기능 구현 및 API 단위 테스트 완료

2) AI 서버 - 백엔드 연동 엔진

- Spring Boot와 FastAPI 간 비동기 통신 구조 구현
- 파일 업로드 → AI 처리 → 결과 반환 → DB 저장 흐름 구축

→ 전체 데이터 흐름 End-to-End 연동 검증 완료

3) 개인화 문제 생성 엔진

- 사용자 정답/오답 데이터를 기반으로 취약 개념 분석 구조 설계
- 취약 개념 기반 문제 재생성 로직 구현

→ Mock 데이터 기반 기능 검증 완료

4) 게임 엔진 연동 (React + Phaser)

- React와 Phaser를 iframe 구조로 분리하여 통합
- 게임 실행 및 반응형 처리 구현

→ 실제 게임 실행 및 UI 동작 검증 완료

5) 시스템 아키텍처 구축

- Frontend-Backend-AI 서버 구조 설계 및 초기 구현
- 핵심 기능 간 데이터 흐름 정의

→ 프로토타입 시스템 통합 완료

2. 그로스

그로스 단계에서는 스타트 단계에서 구현된 엔진을 기반으로 기능 고도화와 AI 파이프라인 설계, 시스템 통합 및 서비스화를 진행하고 있으며, 각 기능별 진척도는 다음과 같다.

1) 전체 시스템 통합 및 배포

- 프론트엔드-백엔드-AI 서버 간 실시간 연동 및 배포 환경 구축

→ 현재 미구현 상태 (진척도 10%)

- 향후 AWS 기반 실제 서비스 배포 및 통합 환경 구축 예정

2) 개인화 분석 및 추천 엔진 고도화

- 학습 데이터 기반 “문제 생성 → 풀이 → 오답 분석 → 재생성” 학습 루프 확장
- ~~취약 개념 기반 추천 알고리즘 정교화~~
- 사용자 오답 패턴 및 문법 태그 기반 취약도 분석을 활용한 문제 Retrieval 및 재생성 로직 고도화

→ 현재 개발 진행 중 (진척도 약 60~70%)

3) AI 문제 생성 엔진 개선

- 문제 유형 다양화 및 난이도 조절 기능 확장
- ~~RAG 및 추가 LLM 적용을 통한 문제 품질 향상~~
- Grammar-aware RAG 구조 및 동적 Prompt 구성을 활용하여 문제 유형 다양화와 문법적 일관성 향상 기능 개발

→ 현재 개발 진행 중 (진척도 약 70~80%)

4) UX 및 게임 시스템 확장

- 다양한 미니게임 템플릿 구현
- 인터랙션 및 보상 시스템(코인, 육성 등) 적용
- 실제 게임 플레이 기반 사용자 경험 검증 완료

→ 구현 완료 (진척도 100%)

G. 최종 목표

- 최종 산출물: AI가 영어 공부 자료를 분석해 초/중등 학생에 대한 맞춤형 게임을 만들어주는 교육용 게임 웹사이트

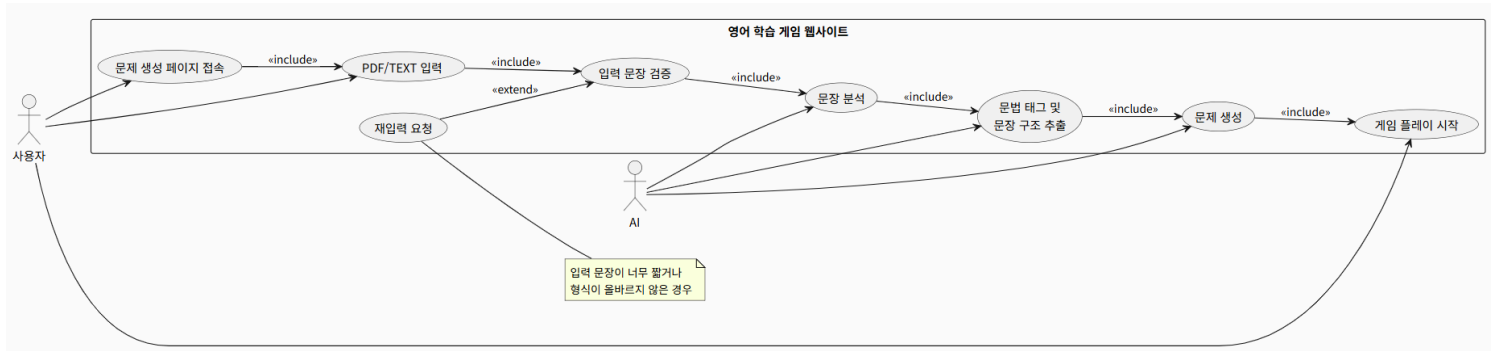
Solution	최종 목표 기능
사용자가 원하는 방식의 맞춤형 게임 제공	● 사용자 요청 시 맞춤형 게임 생성 기능 100% 제공 ● 문제 생성부터 게임 구성까지의 전체 처리 시간 평균 3초 이내, 최대 5초 이내 유지 ● 3가지 문제 유형(O/X, 객관식, 단답형순서 배열) 지원 ● 1회 게임 구성 시 최소 10문제 이상 자동 생성 기능 구현 ● 사용자 입력 데이터 및 학습 방향성(문법, 단어)을 기반으로 개인 맞춤형 문제 생성

다양한 방식의 취약점 분석 및 피드백	• 사용자별 분석 리포트 제공 • 사용자별 취약 문법 태그 도출 기능 구현 • 사용자 게임 결과 기반 오답 유형 분석 정확도 85% 이상 달성 • 최소 3가지 이상의 분석 지표 제공 ◦ 정답률 ◦ 취약 유형 ◦ 취약 문법 • 분석 결과 기반 추천 문제 생성 기능 구현 및 적용
게이미피케이션을 통한 동기 부여 제공	• 미니게임 기반 학습 콘텐츠 최소 6종 이상 구현 • 게임 플레이 결과에 따른 보상 시스템(코인, 점수 등) 구현 • 사용자 평균 플레이 시간 5분 이상 유지 • 사용자 간 공유 기능을 통한 콘텐츠 재사용 및 참여도 증가 구조 구현 • 동기 부여 제공을 위한 육성 게임 기능 제공

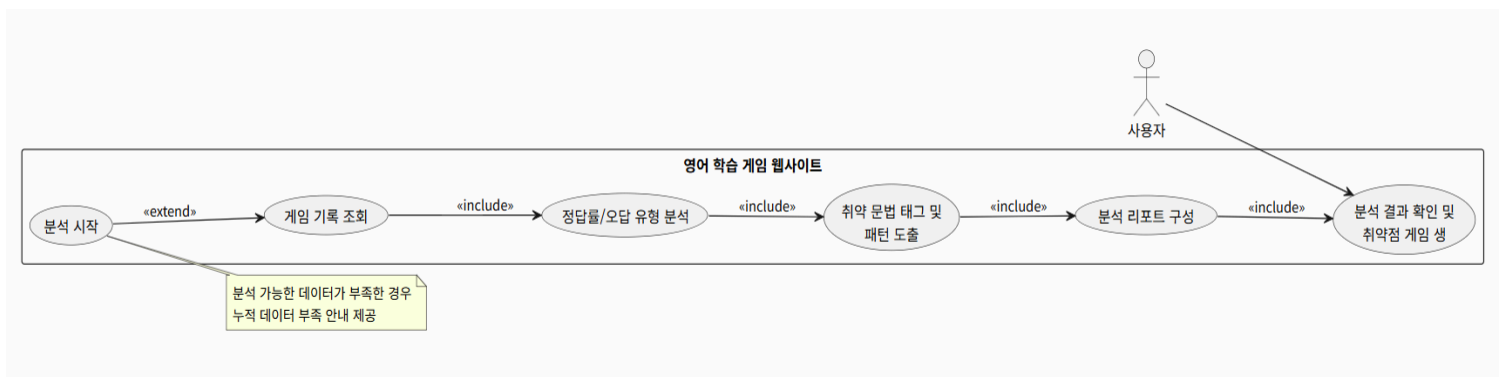
III. Project Design

A. 주요 기능 유스케이스

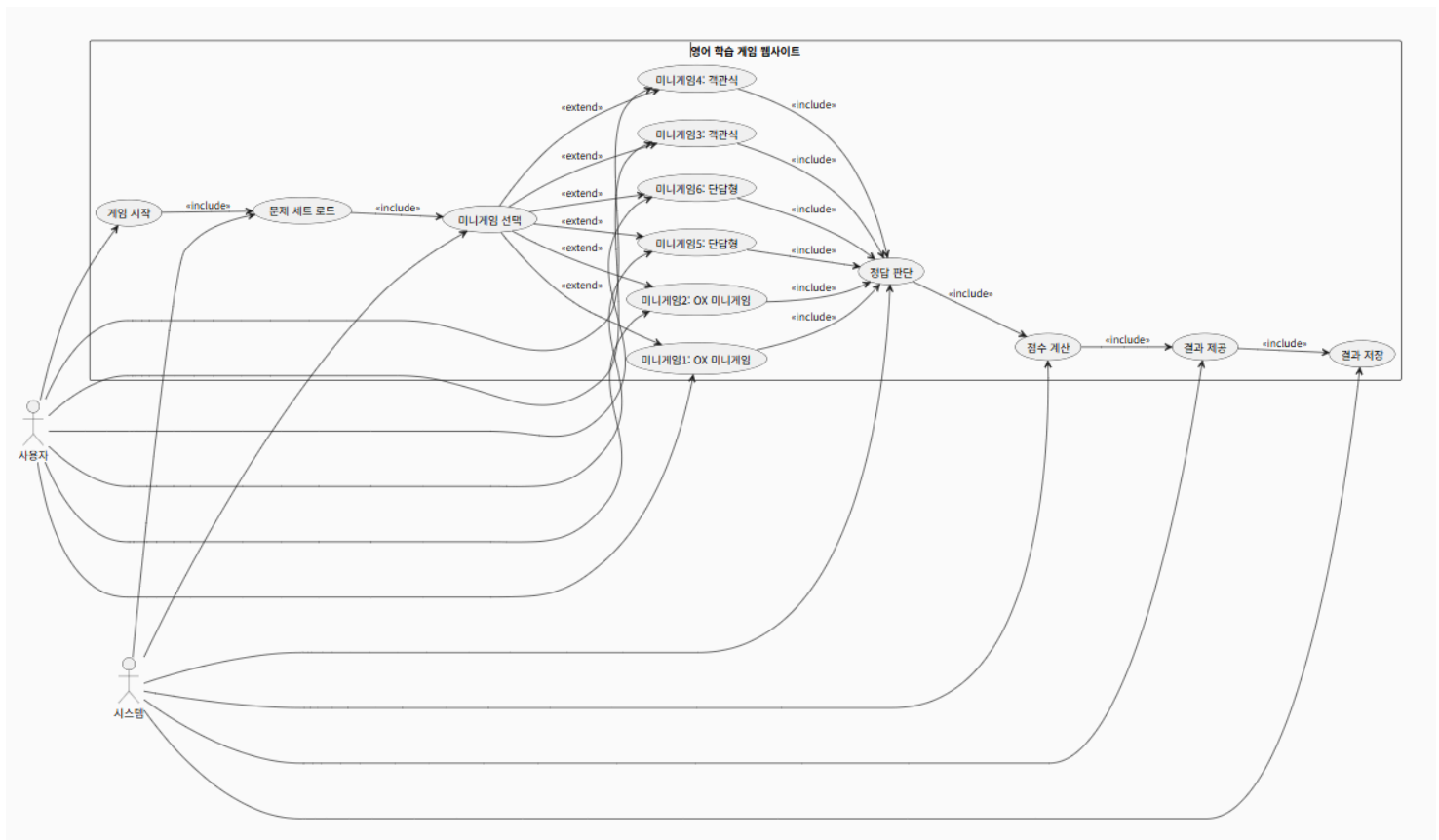
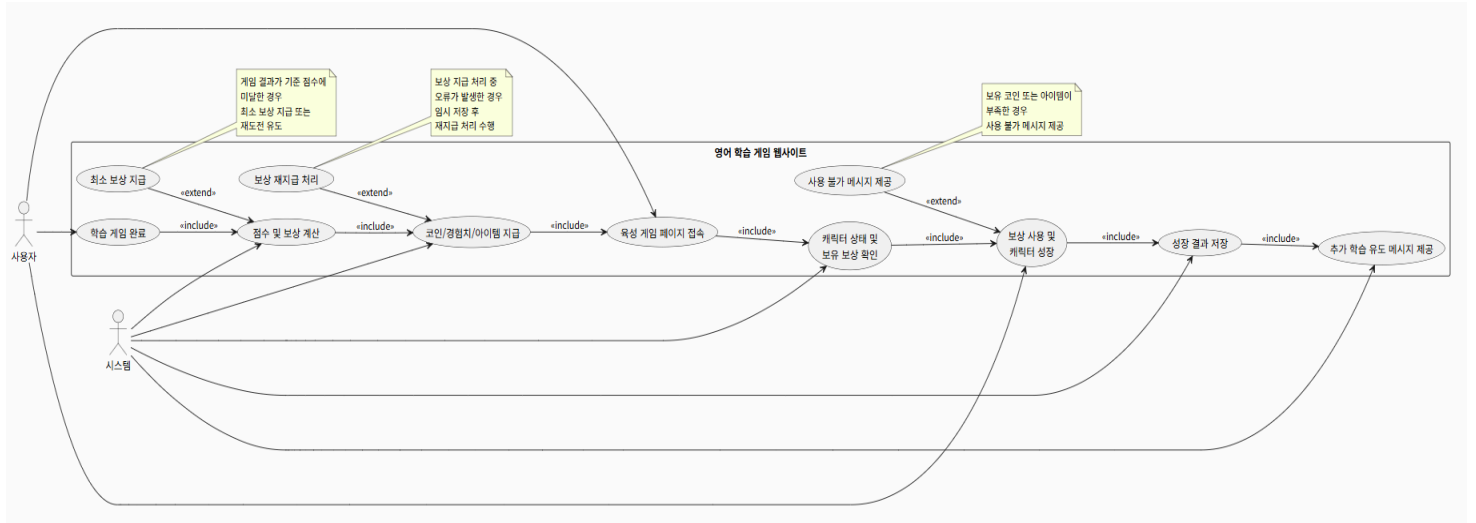
- AI를 통한 문제 생성



- AI를 통한 분석



- 게이미피케이션을 통한 동기 부여 제공



B. 요구사항 정의서

요구 사항 ID	요구사항명	기능 ID	기능명	요구사항 상세 설명	입력 필요 데이터
MEM01	회원가입	MEM01_REGISTER	구글 연동 회원가입 기능	구글 계정과 연동하여 회원 가입할 수 있는 기능	사용자의 구글 계정
		MEM01_INFO	필요한 추가 정보 입력	회원 가입 시 필요한 추가 정보를 사용자에게서 받는 기능	필수: 나이 선택: 생년월일
MEM02	로그인	MEM02_LOGIN	구글 연동 로그인 기능	구글 계정과 연동하여 로그인할 수 있는 기능	사용자의 구글 계정
MEM03	정보 수정	MEM03 MODIFY	사용자 정보 수정 기능	사용자의 입력 정보를 수정할 수 있는 기능	
MAIN01	인삿말	MAIN01_SAY HI	랜덤 인삿말 제공 기능	웹페이지 최초 접속시 랜덤으로 인삿말을 출력해주는 기능	
MAIN02	스트릭	MAIN02_STREAK	스트릭 표시 기능	매일 공부를 수행했는지 아닌지를 확인할 수 있는 스트릭을 출력하는 기능	
MAIN03	취약점	MAIN03_WEAKNESS	취약점 TOP3 표시 기능	사용자의 취약점을 분석해 가장 오답률이 높은 문제 유형 TOP 3를 표시하는 기능	
		MAIN04_WEAKGAME	취약점 관련 게임 생성 버튼 존재	사용자의 취약점을 기반해 게임을 생성할 수 있는 페이지로 연결되는 버튼 기능	
MAIN05	최근 플레이 게임	MAIN05_RECENTGAME	최근 플레이 게임 3~5개 출력	사용자가 최근에 플레이한 게임 3~5개를 출력하고 특정 게임 클릭 시 플레이로 넘어갈 수 있는 기능	
GAME01	게임 만들기	GAME01_OPTION	영단어와 영문법 중 원하는 문제 유형 선택	영단어와 영문법 중 원하는 문제의 유형을 선택할 수 있는 기능	사용자의 영단어, 영문법 중 입력 데이터
		GAME01_SOURCE	문제 제작 소스를 입력	웹페이지에 텍스트 입력 혹은 PDF 파일 업로드를 통해 문제 제작을 위한 소스를 입력할 수 있는 기능	사용자 업로드 PDF 파일 혹은 사용자 입력 텍스트
		GAME01_TITLE	게임의 제목과 설명 입력	게임의 제목과 설명을 입력할 수 있는 기능	필수: 제목 선택: 설명
		GAME01_TYPE	OX, 객관식, 단답형 중 원하는 문제의 유형을 선택	OX, 객관식, 단답형 중 원하는 문제의 유형을 중복 선택할 수 있는 기능	사용자가 선택한 유형 데이터
		GAME01_PREVIEW	학습 목표와 학습 내용 생성	게임 플레이전 학습 목표와 학습 내용을 AI를 통해 생성하는 기능	
PLAY01	게임 플레이	PLAY01_PROBLEM	게임 플레이를 위한 문제 생성	게임 플레이를 위해 풀어야 하는 AI를 활용해 문제를 생성하는 기능	
		PLAY01_ANSWER	각 문제마다 정답 제출	게임 플레이 중 푼 문제에 대한 정답을 제출하는 기능	사용자의 정답 제출 데이터
PLAY02	오답 풀이	PLAY02_WRONG	틀린 문제를 다시 풀기	오답을 조회 후 오답을 다시 풀 수 있는 기능	
		PLAY02_SOLUTION	해설 생성	오답에 대한 해설을 AI를 활용해 생성하는 기능	
PLAY03	게임 결과	PLAY03_RESULT	게임 결과와 해설 조회 및 스트릭 잇기와 코인 보상 얻기	게임 결과와 해설을 조회하고 게임 완료 후 스트릭을 잇고 코인을 받을 수 있는 기능	

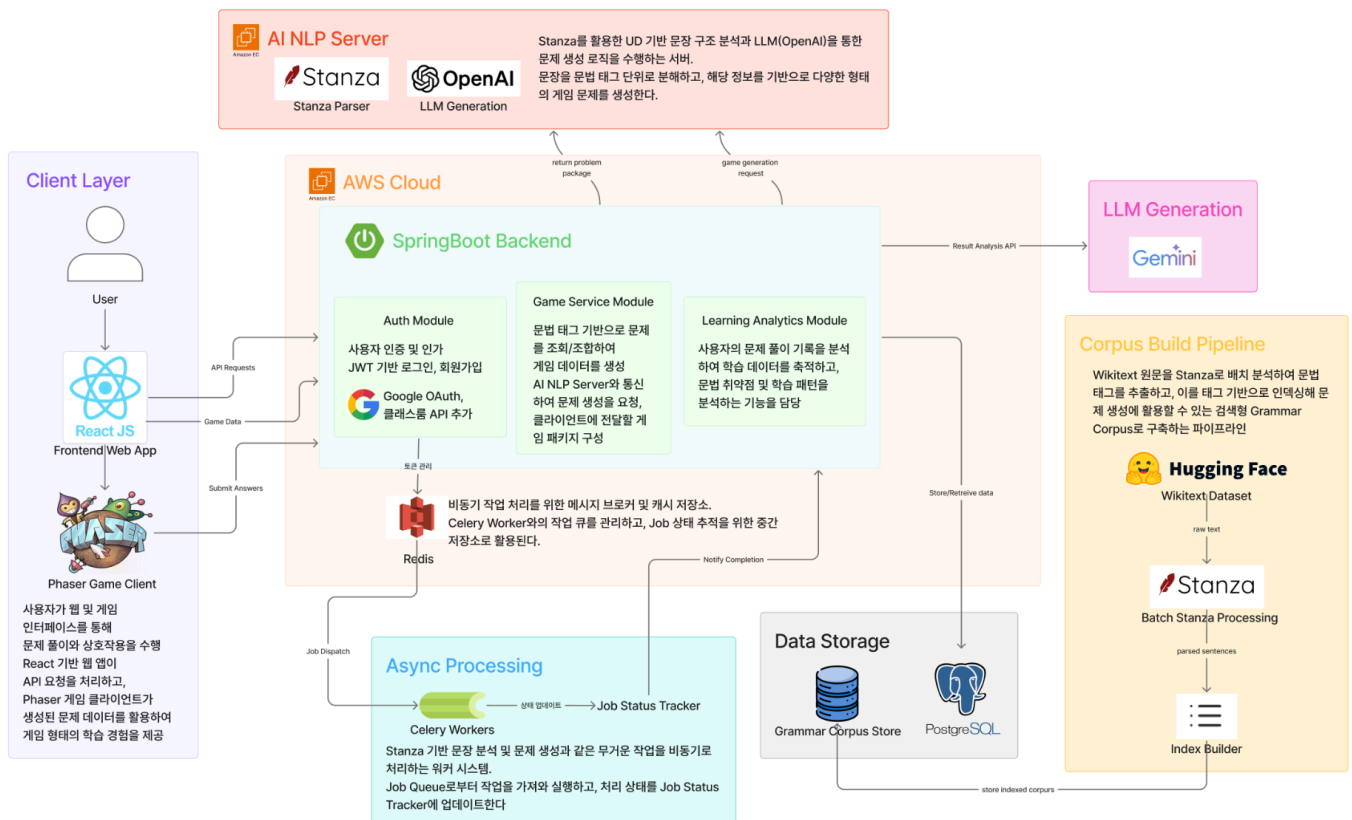
SHARE01	게임 공유	SHARE01_G OOGLE	구글 클래스룸 연동 및 구글 클래스룸을 통한 공유	구글 클래스룸을 연동하여 구글 클래스룸 과제함에 게임 링크를 공유하는 기능	사용자의 구글 클래스룸 사용 동의
		SHARE01_RE AED	사용자들이 공유한 게임을 볼 수 있는 기능	사용자들이 공유한 게임들을 한 눈에 볼 수 있고 클릭 시 그 게임을 할 수 있는 기능	
SHARE01	공개 설정	SHARE_PUBL IC	게임의 공유 범위를 설정하는 기능	게임을 공개로 할 지 비공개로 할 지 사용자가 결정할 수 있는 기능	비공개/공개 설정 데이터
MINI01	육성 게임	MINI01_GAM E	게임에서 얻은 코인을 통해 육성 게임 플레이를 하는 기능	게임에서 얻은 코인을 통해 육성 게임을 플레이 할 수 있는 기능. 간식 주기, 놀아 주기, 밥 주기 등 다양한 옵션을 코인을 통해 선택 가능하다.	
ANAL01	분석	ANAL01_USE R	사용자의 게임 기록을 통해 오답 분석	사용자의 게임 기록 결과를 통해 어떤 문법 태그를 가진 문제를 틀렸는지 시를 통해 분석하여 결과를 표시하는 기능	
		ANAL01_GAM E	사용자의 오답 분석 결과를 통해 오답 문제에 대한 게임 생성	오답 결과를 통해 취약점에 관한 문제를 생성해 게임을 할 수 있는 기능	

C. 전체 시스템 구성

a. SW 구조

1차 보고서에서는 Springboot에서 Gemini를 호출하는 구조가 없었지만, 이후 서비스의 완성도를 높이기 위해 풀이 결과를 기반으로 한 해설 생성 흐름을 구현하고자 하였고, 별도의 LLM Generation(Gemini) 기능을 추가하였다.

LLM Generation 관련 구현 흐름 설명: 사용자가 Phaser 게임 클라이언트에서 문제를 풀이하면, 결과가 postMessage를 통해 React로 전달되고, 이후 Spring Boot Backend로 전송된다. Backend는 사용자 풀이 로그를 저장한 뒤, 비동기적으로 Gemini API를 호출하여 문제 해설 및 결과 분석을 생성한다. 생성된 해설 데이터는 다시 저장되어 사용자 피드백 및 학습 분석 기능에 활용된다.



위 SW는 SpringBoot를 중심으로 사용자의 학습 데이터를 분석하고 AI 기반의 문법 문제를 생성하여 제공하는 언어 학습 게임 플랫폼이다. React와 Phaser 기반의 클라이언트가 사용자에게 게임 형태의 학습 환경을 제공하면, Stanza와 OpenAI가 결합된 NLP 서버가 문장을 정밀 분석하여 맞춤형 문제를 생성한다. 대규모 텍스트 데이터는 Hugging Face와 배치 처리 파이프라인을 통해 문법 코퍼스로 구축되며, 부하가 큰 분석 작업은 Redis와 Celery를 이용한 비동기 방식으로 처리하여 효율성을 높였다. 최종적으로 사용자의 풀이 기록은 Learning Analytics 모듈을 통해 분석되어 PostgreSQL에 저장되며, 전체 시스템은 AWS 클라우드 환경에서 유기적으로 운영된다.

D. 주요 엔진 및 기능 설계

a. Corpus Build Pipeline 모듈

본 모듈은 문법 문제 생성을 위한 데이터 기반을 구축하는 역할을 수행한다.

오픈 코퍼스(Wikitext 등)에서 문장을 수집한 후, 문장 단위로 분리하여 STANZA 기반 자연어 분석을 수행한다. 이후 분석 결과를 기반으로 문법 태그를 생성하고, 이를 PostgreSQL 데이터베이스에 저장하는 구조로 구현하였다.

STANZA 분석 결과로부터 다음과 같은 정보를 추출하였다.

- POS (품사 정보)
- Dependency Parsing (의존 관계)
- Morphological Features (시제, 수, 인칭 등 형태소 특징)

시스템의 핵심은 단순 문장 저장이 아니라, STANZA가 제공하는 POS(품사), Dependency(의존 관계), Morphology(형태소 특징) 정보를 기반으로 “문법 문제 생성 가능 단위”를 정의하는 Rule Layer 설계에 있다.

따라서 이후 분석 결과를 기반으로 문법 태그(Grammar Tag)를 생성하는 Rule Layer를 구현하였다.

즉, “어떤 문법 구조를 어떤 문제 유형으로 변환할 수 있는가”를 규칙 기반으로 정의하였다.

예를 들어 다음과 같은 규칙 기반 매핑을 적용하였다.

- VERB + Tense=Past → 과거시제 문제 후보
- AUX + VERB → 조동사 문제 후보
- nsubj + verb agreement → 주어-동사 수일치 문제 후보
- mark / advcl → 접속사·부사절 문제 후보
- amod / advmod → 형용사·부사 구분 문제 후보

이를 통해 문장을 단순 텍스트가 아닌 “문법적으로 검색 가능한 데이터” 형태로 변환하였다.

또한 PostgreSQL에 문장, 문법 태그, 메타데이터를 함께 저장하여 이후 Retrieval 과정에서 문법 조건 기반 검색이 가능하도록 설계하였다.

현재 기본적인 문법 태그 생성 및 저장 기능은 구현 완료되었으며, 일부 고급 문법 규칙의 세분화 및 태그 정확도 개선은 추가 보완이 필요한 상태이다.

b. AI NLP Server 모듈

AI NLP Server는 사용자 맞춤형 문법 문제를 생성하기 위한 핵심 생성 엔진 역할을 수행한다.

Spring Boot 서버로부터 문제 생성 요청을 전달받으면, 사용자 조건 및 문법 태그를 기반으로 적절한 문장 후보를 Retrieval한 후, LLM API를 통해 최종 문제를 생성하는 구조로 구현하였다.

LLM API와 관련하여, 기존의 단순 생성형 AI 기반 문제 생성 방식은 LLM이 임의의 문장을 생성하기 때문에 문법 오류, 난이도 불일치, 문제 품질 편차 등의 문제가 발생할 수 있다. 이를 해결하기 위해 본 프로젝트에서는 Retrieval-Augmented Generation(RAG) 구조를 적용하였다.

문제 생성 과정은 다음과 같다.

1. 사용자 요청 수집
 - 목표 문법 태그
 - 문제 유형
 - 사용자 취약 문법 정보
 - 난이도 정보
2. Grammar-Aware Retrieval 수행

Corpus Build Pipeline에서 생성한 문법 태그 데이터를 기반으로 PostgreSQL에서 조건에 맞는 문장을 검색한다.

예시:

- tense_past
- subject_verb_agreement
- auxiliary_verb

등의 문법 태그 기반 Retrieval 수행

3. Prompt 구성 (아래 F-a에 더 상세히 서술)

Retrieval된 문장과 문법 태그를 기반으로 LLM 입력 Prompt를 구성하였다.

Prompt에는 다음 정보를 포함하였다.

- 원문 문장
- 문법 태그
- 문제 유형
- 정답 위치
- 출력 JSON 형식

4. LLM 기반 문제 생성

LLM API는 Retrieval된 실제 corpus 문장을 기반으로 객관식, 빈칸형 등의 문제와 해설을 JSON 형태로 생성한다.

예시 출력:

```
{
  "question": "...",
  "choices": ["...", "..."],
  "answer": "...",
  "explanation": "..."
}
```

5. 결과 검증 및 반환

생성 결과는 JSON 형식 검증 후 Spring Boot 서버에 반환되며, 이후 게임 시스템에 전달된다.

본 구조는 단순히 LLM에게 문제 생성을 맡기는 방식이 아니라, STANZA 기반 문법 분석과 Grammar-Aware Retrieval을 결합하여 생성 범위를 제어하도록 설계하였다. 즉, 이미 분석된 corpus 문장을 근거로 문제를 생성하도록 설계하여 문법적 일관성과 신뢰성을 높였다.

또한 AI NLP Server는 독립적인 FastAPI 기반 서버로 분리하여 구현하였으며, 향후 문법 태그 확장 및 문제 유형 다양화가 가능하도록 모듈화 구조를 적용하였다.

c. 게임 문제 풀이 및 비동기 해설 생성 모듈

1) 정의

게임 문제 풀이 및 비동기 해설 생성 모듈은 Phaser에서의 문제 풀이와 React·REST를 잇고, Spring Boot에서 풀이 로그를 저장하며 해설을 조회·생성(Gemini)·보관하는 흐름 전체를 담당한다.

- 학습·게임 진입: React가 JWT와 문제 목록·gameId를 담아 Phaser(iframe)에 START_GAME 메시지를 보내, 게임 루프가 서버가 준 문항으로만 동작하게 한다.
- 풀이 결과 반영: Phaser가 미니게임 결과를 부모에게 postMessage로 알리고, 종료 시 정·오답 목록을 전달한다. React는 종료 이벤트를 받아 분석 UI 등으로 넘긴다.
- 서버 기록·해설: 백엔드는 제출 API로 시도 로그를 남기고, 해설 API로 문항 단위 해설을 조회하거나 없을 때 Gemini로 생성·저장한다.

- 사용자 경험 분리: 게임 중 해설 요청은 응답을 기다리지 않고 진행을 막지 않도록 설계할 수 있고, 종료 후에는 React에서 해설을 모아 보여 주거나 병렬로 미리 요청한다.

2) 사용 프레임워크

구분	프레임워크	사용 방식
게임 실행	Phaser3	2D 미니게임 씬, 사용자 입력
Client UI 및 라우팅	React	게임 페이지 라우팅, iframe으로 Phaser 로드, postMessage·REST 호출·상태 관리
서버	SpringBoot (java 17)	REST API, 트랜잭션, JPA 영속화
데이터	PostgreSQL	문제 풀이 로그, 해설 등 저장
AI	Gemini	문제 해설 생성

3) 구현 설계

- postMessage를 통한 React-Phaser간 데이터 교환 및 규약
 - React → Phaser(iframe): onLoad 직후 Phaser create와의 레이스를 줄이기 위해 지연 재전송과, iframe 쪽 선행 message 리스너로 START_GAME 버퍼링을 둔다.
 - Phaser → React: React는 window message에서 허용 type만 처리해 게임 결과를 받는다.

• 프론트엔드 구현 설계

게임 플레이 단계	전체 화면 iframe에 정적 HTML+Phaser 엔트리를 두고, React는 토큰·문제 배열만 주입한다.
게임 플레이 시 문제 정답 제출 단계 (해설 선요청)	문제의 정답이 제출될 때 비동기로 해설 생성 요청을 수행하고, 게임 로직은 기다리지 않게 하여 Gemini 해설 생성이 지연을 발생하지 않도록 한다.
게임 종료 후 단계	종료 후 생성된 해설을 목록으로 받아 사용자에게 보여준다.

• 백엔드 구현 설계

제출: POST /games/{gameId}/problems/{problemId}/submit + SubmitAnswerRequestDto → ProblemAttempt 누적 저장, gameId·문제 소속 검증, JWT로 사용자 식별.

해설: .../explanation → 기존 Problem.explanation이 있으면 즉시 반환, 없으면 GeminiService → GeminiClient로 생성 후 엔티티에 반영·응답.
 관심사 분리: 제출 경로에는 외부 API가 없어 응답이 가볍고, 해설·Gemini는 별도 엔드포인트에 모은다.

- 데이터 저장 및 보안 구현 설계
 - 영속화
 - 문제 풀이 시도에 대한 객체를 ProblemAttempt 저장
 - 생성된 해설은 문제 객체(Problem)의 필드에 저장
 - 보안
 - 자신이 생성한 문제에 접근 가능하도록 Spring Security에서 인증 예외 경로 외 모든 경로는 로그인 후 이용 가능

E. 주요 기능의 구현

a. Feature 1. 사용자가 원하는 방식의 맞춤형 게임 제공

본 기능은 사용자의 문법 이해도와 학습 패턴을 분석하여, 사용자별 맞춤형 문제 세트를 제공하는 기능이다.

기능 흐름은 다음과 같다.

1. 사용자 입력 및 풀이 데이터 수집
2. STANZA 기반 문법 Parsing 수행
3. 문법 태그 기반 사용자 문법 이해도 벡터 생성
4. Grammar-Aware Retrieval 수행
5. LLM 기반 문제 및 해설 생성
6. 사용자 맞춤형 문제 세트 제공

해당 기능 구현에는 다음 SW 모듈이 사용되었다.

- STANZA
 - 문장을 품사(POS), 의존 관계(Dependency), 형태소 특징(Morphology) 단위로 분석하여 문법 구조를 추출한다.
- Corpus Build Pipeline
 - 문장을 문법 태그 기반 데이터 형태로 변환하고 PostgreSQL에 저장한다.
- PostgreSQL
 - 문장, 문법 태그, 사용자 학습 데이터를 저장한다.
- AI NLP Server
 - 문법 태그 조건 기반 Retrieval과 LLM 문제 생성을 수행한다.
- LLM API
 - Retrieval된 문장을 기반으로 문제와 해설을 생성한다.

사용자가 문제를 풀이하면, 시스템은 사용자의 정답 여부, 오답 패턴, 문제 유형 등을 저장한다. 이후 STANZA 기반 문법 분석 결과를 활용하여 사용자의 문법 이해도를 문법 태그 단위로 수치화하였다.

예를 들어:

- 과거시제 관련 정답률
- 조동사 관련 오답 빈도
- 주어-동사 수일치 오류 빈도

등을 기반으로 사용자별 문법 이해도 벡터를 생성하였다.

이후 AI NLP Server는 해당 정보를 기반으로 Grammar-Aware Retrieval을 수행하여, 사용자가 취약한 문법 태그와 일치하는 corpus 문장을 우선 검색한다.

검색된 문장은 Retrieval 기반 구조(RAG)를 적용하여 LLM API 입력 Prompt에 포함되며, LLM은 Retrieval된 실제 문장을 기반으로 문제와 해설을 생성한다.

b. Feature 2. 다양한 방식의 취약점 분석 및 피드백

본 기능은 사용자의 문제 풀이 결과를 분석하여 반복적으로 취약한 문법 요소를 탐지하고, 해당 문법 유형 중심의 피드백 및 재학습 문제를 제공하는 기능이다.

기능 흐름은 다음과 같다.

1. 사용자 문제 풀이 데이터 수집
2. 정답/오답 기반 문법 태그 분석
3. 취약 문법 패턴 추출
4. 취약 문법 중심 문제 Retrieval 수행
5. LLM 기반 피드백 및 문제 재생성

해당 기능 구현에는 다음 SW 모듈이 사용되었다.

- STANZA
문장 구조 및 문법 요소 분석 수행
- PostgreSQL
사용자별 오답 기록, 문법 태그, 문제 풀이 이력 저장
- Grammar-Aware Retrieval
취약 문법 유형과 일치하는 문장 검색 수행
- AI NLP Server
취약 문법 기반 문제 생성 및 피드백 생성 수행
- LLM API
문법 설명 및 사용자 맞춤형 해설 생성 수행

시스템은 사용자의 문제 풀이 데이터를 기반으로 문법 태그별 정답률 및 반복 오답 패턴을 분석한다.

예를 들어:

- 특정 시제 관련 반복 오답
- 조동사 사용 오류
- 전치사 선택 오류

등을 문법 태그 단위로 분석하여 취약 문법 유형을 추출하였다.

이후 AI NLP Server는 사용자의 취약 문법 태그를 기반으로 Retrieval을 수행하고, 해당 문법 요소를 포함하는 corpus 문장을 검색한다.

검색된 문장은 LLM Prompt에 포함되며, LLM은 이를 기반으로:

- 취약 문법 설명
- 사용자 맞춤형 해설
- 추가 문제

등을 생성한다.

본 기능 역시 단순 생성형 AI 방식이 아니라 Retrieval 기반 생성 구조(RAG)를 적용하여 문법적 정확성과 피드백 품질을 유지하도록 설계하였다.

c. Feature 3. 게이미피케이션을 통한 동기 부여 제공

본 기능은 사용자의 지속적인 학습 참여 유도 및 몰입도 향상을 위해 게이미피케이션을 도입한다고 했을 때, 게임의 원활한 진행을 위해서는 Phaser 게임에서 얻을 수 있는 사용자 문제 풀이 및 정답 데이터를 React를 통해 백엔드로 전달한 뒤, 비동기화식으로 데이터에 대한 해설을 생성할 수 있게 해야 하기 때문에 구현하게 된 기능이다.

기능 흐름은 다음과 같다.

1. 사용자 문제 풀이 및 정답 제출
2. postmessage를 통해 phaser → react 결과 반환
3. react → Spring Boot 풀이 결과 전송
4. 사용자 풀이 로그 저장 및 문제 해설 생성 비동기 호출
5. GEMINI API를 통해 생성된 해설 결과 저장

해당 기능 구현에는 다음 SW 모듈이 사용되었다.

- PHASER
HTML5 게임 엔진을 활용하여 인터랙티브한 2D 웹 게임 제공
- REACT
POSTMESSAGE 를 통해 BE API 호출 응답을 PHASER 로 전송, PHASER <-> BE 통신 담당
- SPRING BOOT
DB 조회 및 저장
- PostgreSQL
사용자별 오답 기록, 문제 풀이 이력 저장
- LLM API
문법 설명 및 사용자 맞춤형 해설 생성 수행

시스템은 Phaser 게임 내에서 발생한 학습 데이터를 React를 통해 Spring Boot로 전달하고, 이를 향후 분석 및 맞춤형 AI 해설 생성에 활용하는 구조를 가진다.

예를 들어, 사용자가 Phaser 게임을 마친 경우 React에 전달되는 학습 데이터는 다음과 같다:

- 정오 여부 : correct
- 정답 여부 및 반응 시간 : responseTimeMs
- 객관식 선택 및 주관식 제출 텍스트 : chosenAnswer, submittedText
- 시도 순번 : attemptOrder
- 기타 : 게임에 속한 정적 문항(지문, 선택지 JSON, 정답, 유형, 범위 등).

이는 React에서 POST /games/{gameId}/problems/{problemId}/submit를 통해 백엔드 서버에 저장된다.

정답 여부, 시도 순서, 선택한 답, 제출한 텍스트 데이터 등 사용자 풀이 로그 데이터들은 이후에 분석·오답 노트·연속 학습 기능을 제공하는데에 활용된다.

이후 시스템은 LLM 모델의 해설 생성으로 인한 불필요한 지연 시간을 막기 위해 해설 생성을 별도의 시간축으로 분리하여 수행하게 되며, 사용자가 해설 요청 시 해당 문제의 엔티티(지문, 선택지, 정답)를 바탕으로 비동기 호출을 수행한다. 방법은 다음과 같다.

- 클라이언트가 해설 API를 호출했을 때, 백엔드 서버의 Problem 객체의 explanation에 값이 없으면 그 요청 처리 중에 Gemini를 호출해 텍스트를 만든 뒤, 같은 트랜잭션 맥락에서 엔티티에 반영하고 반환한다.
- 클라이언트가 해설 API를 호출했을 때, 이미 백엔드 서버의 Problem 객체의 explanation에 값이 있으면 DB에서 즉시 반환한다.

즉, @Async, 메시지 큐, 이벤트 기반 워커를 통해 게임 결과 데이터는 빠르게 백엔드에 제출하고, API boundary를 사용하여 해설 생성을 따로 진행하게끔 했다.

추가로, 생성된 해설 문자열은 Problem 엔티티의 explanation(TEXT) 에 저장되어, 한 번 생성해 두면 같은 문제에 대한 해설 API가 호출되었을 때 빠르게 해설을 제공할 수 있게 해준다.

F. 기타 (AI 설계 상세 내용)

a. Feature 1. 사용자가 원하는 방식의 맞춤형 게임 제공

본 기능에서는 사용자의 문법 이해도와 학습 패턴에 따라 문제를 동적으로 생성하기 위해 LLM API 기반 문제 생성 구조를 구현하였다.

단순 생성형 AI 기반 문제 생성 방식은 모델이 임의의 문장을 생성하기 때문에:

- 문법 오류 발생 가능성
- 난이도 불일치
- 문제 형식 비일관성

- 사용자 수준과 맞지 않는 문제 생성

등의 문제가 발생할 수 있다.

이를 해결하기 위해 본 프로젝트에서는 Retrieval-Augmented Generation(RAG) 구조를 적용하여, LLM이 이미 분석된 corpus 문장을 기반으로 문제를 생성하도록 설계하였다.

AI NLP Server는 Retrieval 결과와 사용자 학습 상태를 기반으로 Prompt를 동적으로 구성하도록 구현하였다.

실제 Prompt:

```

You are an English grammar problem generator.

Generate one {question_type} grammar question based on the following sentence.

Sentence:
"{retrieved_sentence}"

Grammar Tag:
{grammar_tag}

Difficulty:
{difficulty}

Requirements:
- Focus on the target grammar tag
- Generate {choice_count} choices
- Only one correct answer
- Return JSON format only

Output Format:
{
  "question": "...",
  "choices": ["...", "...", "...", "..."],
  "answer": "...",
  "explanation": "..."
}

```

실제 문제 생성 시에는:

- {retrieved_sentence}
- {grammar_tag}
- {question_type}
- {difficulty}
- {choice_count}

등의 값이 Retrieval 결과와 사용자 학습 상태에 따라 동적으로 삽입되도록 구현하였다.

예를 들어 사용자가 과거시제 문제에 반복적으로 취약한 경우에는 grammar_tag 값이 tense_past로 설정되며, 사용자의 학습 난이도가 높은 경우에는 상대적으로 복잡한 문장을 Retrieval하도록 설계하였다. 또한 객관식 문제 생성 요청 시에는 question_type 값이 multiple_choice로 설정되는 방식으로 Prompt가 구성된다.

또한 LLM API의 출력은 반드시 JSON 형식으로 반환되도록 설계하였다.

예시 출력:

```
{
  "question": "Choose the correct verb form.",
  "choices": [
    "go",
    "goes",
    "went",
    "going"
  ],
  "answer": "went",
  "explanation": "The sentence describes a past event, so the past tense form is"
}
```

AI NLP Server에서는 LLM 생성 결과를 그대로 사용하는 것이 아니라, 생성된 응답에 대해 JSON 형식 검증 및 응답 구조 검사를 수행하도록 구현하였다. 특히 필수 필드 존재 여부, 선택지 개수, 응답 형식의 정상 여부 등을 검사하며, 비정상적인 응답이 생성될 경우 예외 처리 후 재생성하거나 오류 응답을 반환하도록 설계하였다. 이를 통해 생성 결과를 Spring Boot 서버에서 안정적으로 처리할 수 있도록 구현하였다.

이를 통해 단순 ChatGPT 기반 생성이 아니라, Retrieval 기반 생성 범위 제어와 출력 형식 검증을 결합한 안정적인 AI 문제 생성 구조를 구현하였다.

b. Feature 2. 다양한 방식의 취약점 분석 및 피드백

본 기능에서는 사용자의 반복 오답 패턴과 취약 문법 요소를 분석하여, 사용자 맞춤형 피드백 및 재학습 문제를 생성하기 위해 LLM 기반 피드백 생성 구조를 구현하였다.

시스템은 사용자의 문제 풀이 결과를 문법 태그 단위로 분석하며:

- 특정 시제 관련 반복 오답
- 조동사 사용 오류
- 전치사 선택 오류
- 주어-동사 수일치 오류

등의 취약 문법 패턴을 추출하도록 설계하였다.

이후 AI NLP Server는 사용자의 취약 문법 유형과 관련된 Retrieval 결과를 기반으로 Prompt를 동적으로 구성하여 LLM API를 호출한다.

실제 Prompt:

```
You are an English grammar tutor.

The user frequently makes mistakes in {weak_grammar_tag}.

Sentence:
"{retrieved_sentence}"

Grammar Tag:
{grammar_tag}

Requirements:
- Explain the grammar mistake clearly
- Provide the corrected sentence
- Generate one additional practice question
- Return JSON format only
```

실제 피드백 생성 시에는:

- {weak_grammar_tag}
- {retrieved_sentence}
- {grammar_tag}

등의 값이 사용자 오답 패턴 분석 결과에 따라 동적으로 삽입되도록 구현하였다.

예를 들어 사용자가 주어-동사 수일치 문제를 반복적으로 틀리는 경우에는 weak_grammar_tag 값이 subject_verb_agreement로 설정되며, 전치사 관련 정답률이 낮은 경우에는 weak_grammar_tag 값이 preposition으로 설정되는 방식으로 Prompt가 구성된다.

LLM은 Retrieval된 문장을 기반으로 문법 설명, 오답 원인 분석, 올바른 문장 제시 및 추가 연습 문제 등을 생성하도록 설계하였다.

예시 출력:

```
{
  "mistake": "subject-verb agreement",
  "correction": "He goes to school every day.",
  "explanation": "Third-person singular subjects {...}",
  "practice_question": "She ____ to school every morning."
}
```

또한 Feature 1과 동일하게 생성 결과에 대한 JSON 형식 검증, 응답 구조 검사, 필수 필드 존재 여부 검사 및 비정상 응답 처리 등의 후처리를 수행하여 생성 결과의 안정성을 유지하도록 구현하였다.

본 기능은 단순 자동 채점 기능이 아니라, 사용자의 취약 문법 유형을 분석하고 해당 문법 중심의 피드백과 재학습 문제를 동적으로 생성하는 AI 기반 학습 지원 시스템으로 구현하였다.

c. Feature 3. Gemini API를 이용한 해설 생성

본 기능에서는 사용자가 게임 오답 풀이에서 문제 해설 필요할 때, 서버가 Google Gemini API를 호출해 문제에 대한 해설 텍스트를 생성하도록 LLM 연동 구조를 구현하였다.

해설 생성을 위해 필요한 속성은 다음과 같다.

- 지문: question
- 객관식일 경우 선택지 목록: optionsJson을 파싱한 리스트
- 정답 문자열: correctAnswer

이후 GeminiService는 위 값들을 조합해 자연어 프롬프트를 구성하고, 내부에서 GeminiClient가 Gemini REST generateContent API를 호출한다.

```
public String generateExplanation(String question, List<String> options, String correctAnswer) {
    StringBuilder prompt = new StringBuilder();
    prompt.append(str: "다음 퀴즈 문제에 대한 정답 해설을 작성해 주세요. 학습자가 왜 이 정답이 맞는지, 오답이 왜 틀렸는지 이해할 수 있도록 2~5문장으로 설명해 주세요.");
    prompt.append(str: " 풀이 과정이나 근거를 간단히 포함해 주세요.\n");
    prompt.append(str: " 【문제】 \n").append(question).append(str: "\n\n");
    if (options != null && !options.isEmpty()) {
        prompt.append(str: " 【선택지】 \n").append(String.join(delimiter: "\n", options)).append(str: "\n\n");
    }
    prompt.append(str: " 【정답】 \n").append(correctAnswer).append(str: "\n\n 【해설】 \n");
    String raw = geminiClient.generateText(prompt.toString());
    return (raw != null && !raw.isBlank()) ? raw.trim() : "(해설 생성 실패)";
}
```

프롬프트는 지시문 + 【문제】 / 【선택지】 (있을 때만) / 【정답】 / 【해설】 블록으로 이어지며, 객관식이면 선택지 줄이 String.join을 통해 삽입된다.

```
Map<String, Object> body = Map.of(
    k1: "contents", List.of(Map.of(k1: "parts", List.of(Map.of(k1: "text", prompt)))),
    k2: "generationConfig", Map.of(
        k1: "temperature", v1: 0.7,
        k2: "maxOutputTokens", v2: 8192
    )
);
```

LLM 호출 및 요청 본문은 GeminiClient에서 WebClient로 JSON 바디를 만들어 전송한다. contents[.].parts[.].text에 위에서 만든 프롬프트 문자열이 들어가고, generationConfig로 온도·최대 출력 토큰을 지정한다.

```
{
  "candidates": [
    {
      "content": {
        "role": "model",
        "parts": [
          {
            "text": "The correct answer is (B) 'has lived'. We use the present perfect (have/has + past participle) to"
          }
        ]
      },
      "finishReason": "STOP",
      "index": 0
    }
  ],
  "usageMetadata": {
    "promptTokenCount": 210,
    "candidatesTokenCount": 120,
    "totalTokenCount": 330
  }
}
```

위 JSON은 Gemini 호출 응답 예시이다. 응답 처리에서 구조화된 JSON 스키마를 강제하지 않아, 해설 응답을 일반 텍스트를 받는다. 대신 안정성을 위해 아래와 같은 후처리 검증을 수행한다.

1. HTTP 쿼터: 서버가 Gemini에 요청을 보낸 뒤, 오류 발생 시 해설 생성이 실패한다. 응답이 할당량 초과(429)일 때는 쿼터 초과로 구분해 처리한다.
2. 응답 JSON 구조: API가 돌려준 본문 JSON 안에 답변 목록이 있는지 확인하고, 답변이 없거나 구조가 맞지 않아 읽을 수 없으면 빈 문자열로 간주한다.
3. 생성 텍스트 후처리: 해설의 앞뒤 공백을 정리한 뒤 내용이 없으면, 해설 생성 실패로 간주하고 에러 처리 후 fail message를 반환한다.

```
if (problem.getExplanation() != null && !problem.getExplanation().isBlank()) {
    return problem.getExplanation();
}
List<String> options = parseOptionsJson(problem.getOptionsJson());
String generated = geminiService.generateExplanation(problem.getQuestion(), options, problem.getCorrectAnswer());
problem.setExplanation(generated);
return generated;
```

해설 조회 시 이미 Gemini를 통해 만들어둔 해설 text 값이 있으면 DB만 읽고, 값이 없을 때만 위 파이프라인을 태운 뒤 같은 엔티티 필드에 저장해 동일 문항 해설 재요청 시 재생성을 피하도록 설계하였다.